

Introduction à l'algorithmique et à la
programmation

Les tests unitaires

Les tests unitaires

Un test unitaire, c'est quoi ?

- O Une petite démonstration vaut mieux qu'un long discours
...<https://www.youtube.com/watch?v=XrXkbnfCYzU>
- O Le débogage du code :
 - 50% du temps d'un expert
 - 90% du temps d'un débutant
 - Tests : Réduire le temps de test et augmenter le temps de mise au point d'algorithme
- O Testeur : c'est un métier à part entière :
 - But : Détecter les anomalies d'un programme (logiciel)
 - Établir un protocole pour vérifier que toutes les fonctionnalités sont présentes

Les tests unitaires

Que tester ?

- O Éviter les tests triviaux : perte de temps
- O Tester chaque procédure et fonction (la réussite, l'échec et la cohérence)
- O Il est important de connaître ses points faibles
- O Il existe des catalogues d'astuces
- O Il existe des méthodes et techniques particulières.

Les tests unitaires

Exemple : tests de la fonction de recherche du maximum de 3 entiers

```
// prototype de la fonction à tester
int maximum(int nb1, int nb2, int nb3);
// prototype de la fonction de test
int un_tu_max_3_entiers(int e1, int e2, int e3, int sortie_attendue);
// prototype de la fonction pour plusieurs tests
int des_tu_max_3_entiers();

// le programme principal
int main(){
    printf("resultat des tests unitaires %d\n", des_tu_max_3_entiers());
    return EXIT_SUCCESS;
}
// définition de la fonction pour un test :
int un_tu_max_3_entiers(int e1, int e2, int e3, int sortie_attendue){
    int ok = 0;
    if (maximum(e1, e2, e3) == sortie_attendue){
        ok = 1;
    }
    return ok;
}
```

Les tests unitaires

```
// définition de la fonctions de test multiples
int des_tu_max_3_entiers(){
    int ok = 0;
    if (    un_tu_max_3_entiers(0,0,0,0) &&
          un_tu_max_3_entiers(3,2,1,3) &&
          un_tu_max_3_entiers(2,3,1,3) &&
          un_tu_max_3_entiers(1,2,3,3) &&
          un_tu_max_3_entiers(2,1,3,3) &&
          un_tu_max_3_entiers(-3,-2,-1,-1)){
        ok = 1;
    }
    return ok;
}

// definition de la fonction à tester
int maximum(int e1, int e2, int e3){
    ...
    return max;
}
```

Les tests unitaires

Exemple : tests de la fonction factorielle :

```
// prototype de la fonction à tester
int factorielle(int a);
// prototype de la fonction de test
int un_tu_factorielle(int entree, int sortie_attendue);
// prototype de la fonction pour plusieurs tests
int des_tu_factorielle();
// le programme principal
int main(){
    printf("resultat des tests unitaires %d\n", des_tu_factorielle());
    return EXIT_SUCCESS;
}
// définition de la fonction pour un test :
int un_tu_factorielle(int entree, int sortie_attendue){
    int ok = 0;
    if (factorielle(entree) == sortie_attendue){
        ok = 1;
    }
    return ok;
}
```

Les tests unitaires

// définition de la fonctions de test multiples

```
int des_tu_factorielle(){  
    int ok = 0;  
    if (    un_tu_factorielle(0,1) &&  
          un_tu_factorielle(1,1) &&  
          un_tu_factorielle(5,120)){  
        ok = 1;  
    }  
    return ok;  
}
```

// definition de la fonction à tester

```
int factorielle(int a){  
    ...  
    return res;  
}
```
